Spectral Clustering: Theory, Algorithm, and Experiments

1 Introduction

Spectral clustering is a graph-based clustering method that is especially attractive when cluster geometry is non-convex, curved, or better described by local neighborhood relations than by Euclidean centroids. The basic move is to convert data into a weighted graph, study the graph Laplacian, extract a small number of eigenvectors, and run a simple clustering method in that spectral embedding. We focus on the normalized spectral clustering algorithm of Ng, Jordan, and Weiss (NJW) [**?**], while relating it to the normalized-cut objective of Shi and Malik [**?**] and the broader graph-Laplacian view in von Luxburg's tutorial [**?**].

The report combines three complementary pieces: a compact theoretical account of normalized spectral clustering, a Python implementation with explicit computational analysis, and an empirical study across synthetic, vector, and graph datasets.

2 Theory

2.1 Graph construction and Laplacians

Given data points $x_1, \ldots, x_n \in \mathbb{R}^d$, we build a weighted undirected graph with affinity matrix $W \in \mathbb{R}^{n \times n}$, where W_{ij} is large when x_i and x_j are similar. Let D be the diagonal degree matrix, $D_{ii} = \sum_j W_{ij}$. The two most common Laplacians are the unnormalized Laplacian

$$L = D - W$$

and the symmetric normalized Laplacian

$$L_{\text{sym}} = I - D^{-1/2} W D^{-1/2}.$$

When the graph has k connected components, L_{sym} has exactly k zero eigenvalues, and the associated eigenvectors are piecewise constant on those components. In the nearly disconnected case, the bottom k eigenvectors remain approximately piecewise constant, which is the spectral signal exploited by clustering algorithms [**?**].

2.2 From normalized cut to an eigenproblem

For a two-way partition $(A, \bar{A})$, the cut is

$$\text{cut}(A, \bar{A}) = \sum_{i \in A,\, j \in \bar{A}} W_{ij}.$$

Minimizing cut alone is degenerate because it prefers isolating tiny sets. Shi and Malik proposed the normalized cut objective

$$\mathrm{Ncut}(A, \bar{A}) = \frac{\mathrm{cut}(A, \bar{A})}{\mathrm{vol}(A)} + \frac{\mathrm{cut}(A, \bar{A})}{\mathrm{vol}(\bar{A})}, \qquad \mathrm{vol}(A) = \sum_{i \in A} D_{ii},$$

which balances low inter-cluster connectivity against cluster size [**?**]. The exact optimization is discrete and NP-hard in general, so spectral clustering uses a continuous relaxation. That relaxation leads to a generalized eigenvalue problem and, after normalization, to the bottom eigenvectors of a Laplacian matrix. This is the main mathematical reason spectral clustering works: difficult discrete partitioning is replaced by a tractable linear algebra problem [**?**, **?**].

2.3 Why NJW?

There are several common flavors of spectral clustering. Unnormalized methods work directly with $L = D - W$. Shi-Malik normalized cuts solve a generalized eigenproblem. NJW instead computes the top eigenvectors of a normalized affinity-like matrix, or equivalently the bottom eigenvectors of a normalized Laplacian, then normalizes rows and applies k-means [**?**]. We chose NJW because it is simple, standard, easy to implement in Python, and closely tied to the normalized-cut relaxation while remaining computationally clean.

3 Algorithm

3.1 Implemented method

Our implementation uses an m-nearest-neighbor graph with RBF weights:

$$W_{ij} = \exp\bigl(-\gamma \|x_i - x_j\|_2^2\bigr)$$

when x_j is among the m nearest neighbors of x_i. The scale parameter γ is chosen from the median nonzero neighbor distance. The graph is then symmetrized.

Input. Data matrix $X \in \mathbb{R}^{n \times d}$, number of clusters k, number of neighbors m.

Output. Predicted cluster labels $\hat{y}_1, \ldots, \hat{y}_n$.

Steps.

1. Build a sparse weighted m-nearest-neighbor graph.
2. Form $L_{\mathrm{sym}} = I - D^{-1/2} W D^{-1/2}$.
3. Compute the k eigenvectors associated with the smallest eigenvalues of L_{sym}.
4. Stack them into $U \in \mathbb{R}^{n \times k}$ and row-normalize to obtain Y.
5. Run k-means on the rows of Y.

3.2 Computational cost

There are three main costs.

1. **Graph construction.** Exact pairwise distances would cost $O(n^2 d)$, but the stored sparse k-NN graph has only $O(mn)$ nonzeros.

2. **Eigensolve.** This is the main bottleneck. Dense eigendecomposition is cubic in n, while sparse iterative eigensolvers scale much better in practice and depend strongly on the number of graph edges and the spectral gap [**?**, **?**].

3. **k-means in spectral space.** Once the embedding is computed, the final step is cheap because the embedding has only k columns.

We measured these costs directly. On a two-moons benchmark, increasing n from 150 to 900 increased the spectral pipeline runtime from about 0.012 s to 0.220 s, while the plain k-means baseline remained around 0.009–0.012 s. The eigensolver dominated the growth, while graph memory rose from about 24 KiB to 161 KiB for the sparse affinity matrix. Figure **??** shows these trends. Exact timings vary by machine, so we interpret the trend rather than the individual seconds.

4 Experimental design

4.1 Evaluation metrics

For labeled benchmarks we report accuracy after optimal label matching, adjusted Rand index (ARI), normalized mutual information (NMI), and purity.

Because cluster labels are permutation-invariant, accuracy is computed only after solving the best one-to-one assignment between predicted cluster IDs and ground-truth classes:

$$\mathrm{Acc}(y, \hat{y}) = \frac{1}{n} \max_{\pi \in S_k} \sum_{i=1}^{n} \mathbf{1}\{y_i = \pi(\hat{y}_i)\}.$$

Here S_k denotes the set of permutations of the cluster labels.

ARI compares two partitions through pairwise co-membership counts and adjusts for the agreement expected by chance. Using the standard contingency-table notation, it can be written as [**?**]

$$\mathrm{ARI} = \frac{\sum_{ij} \binom{n_{ij}}{2} - \dfrac{\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}}{\binom{n}{2}}}{\dfrac{1}{2}\left[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}\right] - \dfrac{\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}}{\binom{n}{2}}}.$$

A value near 0 indicates roughly chance-level agreement, and 1 indicates identical partitions.

NMI measures the information shared by two partitions:

$$\mathrm{NMI}(U, V) = \frac{I(U; V)}{\mathrm{avg}\big(H(U), H(V)\big)},$$

where $I(U; V)$ is mutual information and $H(\cdot)$ is entropy. The normalized version lies in $[0, 1]$ and is invariant to label permutations [**?**].

Purity assigns each predicted cluster to its majority true label and reports the resulting fraction of correctly assigned points:

$$\mathrm{Purity}(\mathcal{C}, \mathcal{G}) = \frac{1}{n} \sum_{C \in \mathcal{C}} \max_{G \in \mathcal{G}} |C \cap G|.$$

In the discussion below, ARI is the most informative single metric because it is sensitive to global partition structure rather than only majority assignment.

4.2 Datasets

We used the following groups of experiments.

- **Synthetic success cases:** two moons, concentric circles, Gaussian blobs, high-dimensional moons (moons embedded in 20 dimensions), and high-dimensional blobs.
- **Synthetic stress cases:** overlapping blobs and a bridge-connected two-cluster dataset.
- **Real/vector datasets:** Iris and four digit experiments: $\{0, 1\}$, $\{1, 7\}$, $\{0, 1, 3, 6\}$, and all ten classes. We use `sklearn.datasets.load_digits`, which provides a compact and reproducible benchmark [**?**].
- **Graph datasets:** Zachary's karate club and a stochastic block model benchmark.
- **Robustness and scaling:** a Gaussian-noise sweep on two moons, and a runtime/memory benchmark as n increases.

For graph datasets, plain feature-space k-means is not naturally defined. To keep the comparison explicit, we used a weak but transparent baseline: k-means on adjacency rows. This is not a state-of-the-art graph clustering baseline, but it provides a concrete comparator.

5 Results: synthetic data

Table **??** summarizes the main synthetic experiments. The expected pattern is visible on the clean low-dimensional examples: spectral clustering strongly beats k-means on non-convex geometry (two moons, circles), and ties or nearly ties on well-separated blobs. The stress cases are more nuanced. On high-dimensional moons, spectral clustering has a modest quantitative advantage, but the 2D projection in Figure **??** makes clear that neither method cleanly recovers the original moon structure after many noisy coordinates are added. On the bridge-connected dataset, the method is better described as partially degraded rather than catastrophically failed: it separates the two main masses reasonably, but the thin bridge lowers the ARI and weakens the graph-cut interpretation.

Table 1: Synthetic experiments. ARI is reported for both methods.

Dataset	Spectral ARI	k-means ARI	Comment
Two moons	1.000	0.245	Canonical non-convex success case.
Circles	1.000	-0.002	Spectral cleanly separates rings.
Blobs	0.990	0.995	Both methods work on convex, separated clusters.
High-dim moons	0.404	0.259	Both methods degrade; spectral keeps a modest numerical edge.
High-dim blobs	1.000	1.000	Both methods remain strong.
Overlapping blobs	0.302	0.254	Ambiguous partition; both methods degrade.
Bridge stress	0.552	0.532	Thin bridge partially weakens the intended cut.

Figures **??**–**??** show representative examples. Spectral clustering does exactly what we expect on two moons and concentric circles. The high-dimensional moons and bridge examples are intentionally less clean: the metrics show only partial advantages, and the plots should be read as stress tests rather than visual proofs of successful recovery.

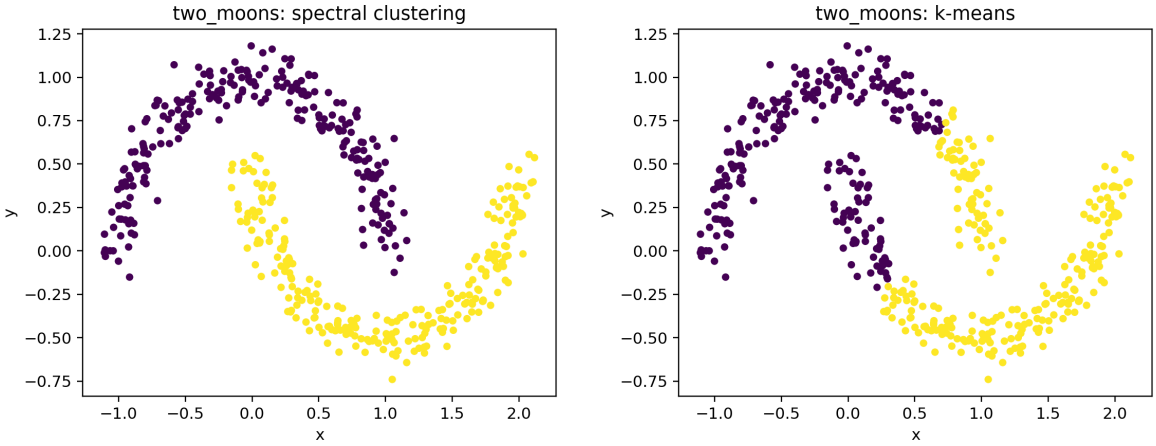

Figure 1: Two moons in original 2D coordinates (x, y). Spectral clustering respects the non-convex geometry, while k-means imposes a centroid-based split.

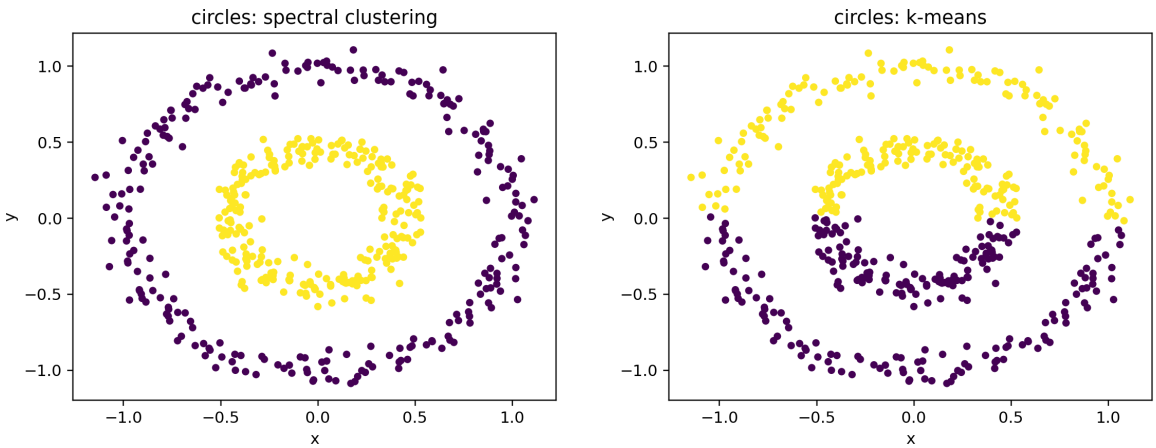

Figure 2: Concentric circles in original 2D coordinates (x, y). Spectral clustering separates the rings; k-means prefers a radial partition.

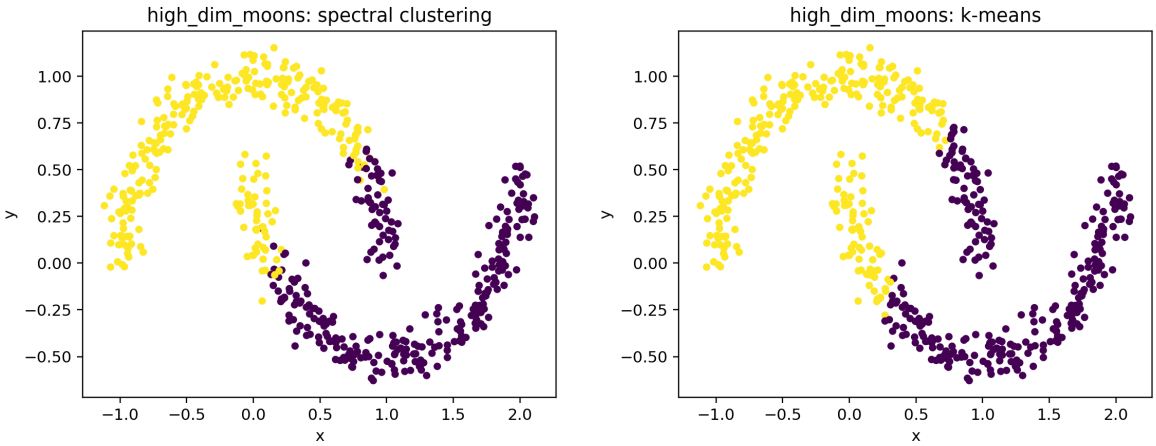

Figure 3: Moons embedded in higher dimension using additional noise coordinates, plotted here using the first two coordinates. Both methods degrade; spectral clustering retains a modest quantitative advantage but no longer cleanly recovers the moon structure in this view.

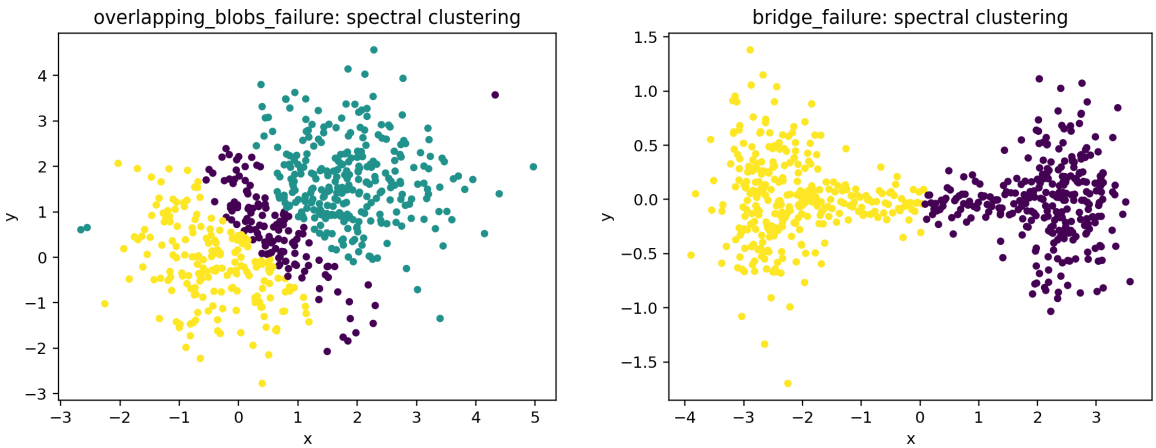

Figure 4: Two stress cases in original 2D coordinates (x, y): overlapping blobs (left) and a bridge-connected dataset (right). In the bridge example, spectral clustering still separates the main masses, but the thin connection weakens the intended cut and reduces ARI.

The bridge-connected example is also a useful reminder that a visually plausible 2D partition need not yield a high pairwise clustering score. In this dataset the two large endpoint masses are separated reasonably well, but the bridge region creates many ambiguous assignments near the connection. Because ARI evaluates agreement at the level of point pairs rather than only the gross shape of the partition, errors on a relatively small set of bridge points can induce a much larger number of pairwise disagreements. The scatter plot should therefore be read as evidence of partial geometric recovery rather than near-perfect agreement with the reference labels.

6 Results: real data and digit subsets

We study four digit subsets of increasing difficulty: $\{0, 1\}$, $\{1, 7\}$, $\{0, 1, 3, 6\}$, and all digits. This progression makes it possible to distinguish regimes where local graph structure is helpful from regimes where raw pixel-space geometry already favors centroid-based clustering. Table **??** reports those results. The outcome is deliberately presented as a mixed result rather than a blanket win for spectral clustering. Spectral clustering is excellent on $\{0, 1\}$ and strong on all ten digits, but it is slightly worse than k-means on $\{1, 7\}$ and clearly worse on $\{0, 1, 3, 6\}$ for the particular k-NN graph used here. Our hypothesis is that raw pixel-space Euclidean distances already separate some digit subsets nearly linearly, while the local graph can become brittle when classes have different stroke styles and densities.

Table 2: Digit-subset experiments.

Subset	Method	Accuracy	ARI	Purity
$\{0, 1\}$	Spectral	0.994	0.978	0.994
$\{0, 1\}$	k-means	0.994	0.978	0.994
$\{1, 7\}$	Spectral	0.997	0.989	0.997
$\{1, 7\}$	k-means	1.000	1.000	1.000
$\{0, 1, 3, 6\}$	Spectral	0.780	0.689	0.780
$\{0, 1, 3, 6\}$	k-means	0.935	0.838	0.935
All digits	Spectral	0.769	0.680	0.784
All digits	k-means	0.589	0.468	0.623

Figure **??** shows the $\{0, 1, 3, 6\}$ subset after PCA to two dimensions. This plot is illustrative only: it compresses a 64-dimensional pixel space into two axes and therefore cannot fully explain the metric gap in Table **??**. The stronger evidence is the quantitative table and the purity summary in Figure **??**.

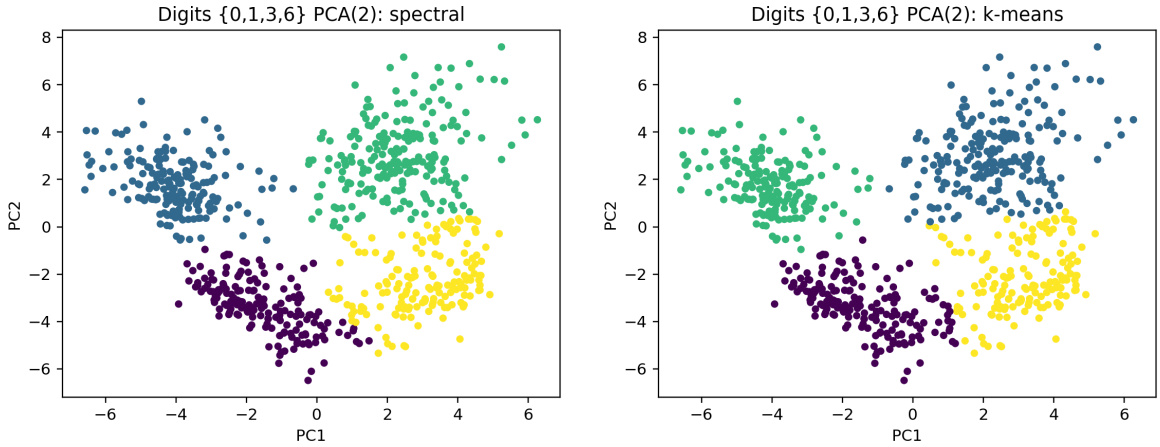

Figure 5: Digits subset $\{0, 1, 3, 6\}$ visualized in the first two PCA coordinates $(\mathrm{PC1}, \mathrm{PC2})$. Both clusterings look fairly clean in this projection; the quantitative table is needed to evaluate the actual clustering quality.

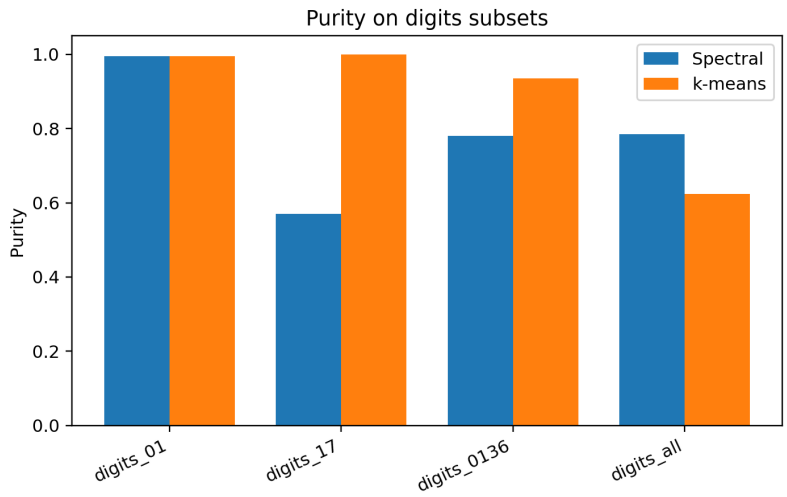

Figure 6: Purity on the four digit experiments.

On Iris, spectral clustering also improves over k-means: ARI rises from 0.620 to 0.707. This is a more modest gain than on clearly non-convex synthetic data, which matches expectations.

7 Results: graph data

Spectral clustering is especially natural on graph data. On Zachary's karate club graph, spectral clustering reaches accuracy 0.971 and ARI 0.882, far above the adjacency-row k-means baseline (accuracy 0.676, ARI 0.109). On a planted stochastic block model, spectral clustering perfectly recovers the communities, while adjacency-row k-means is also strong but slightly worse. These two experiments strengthen the community-detection side of the study.

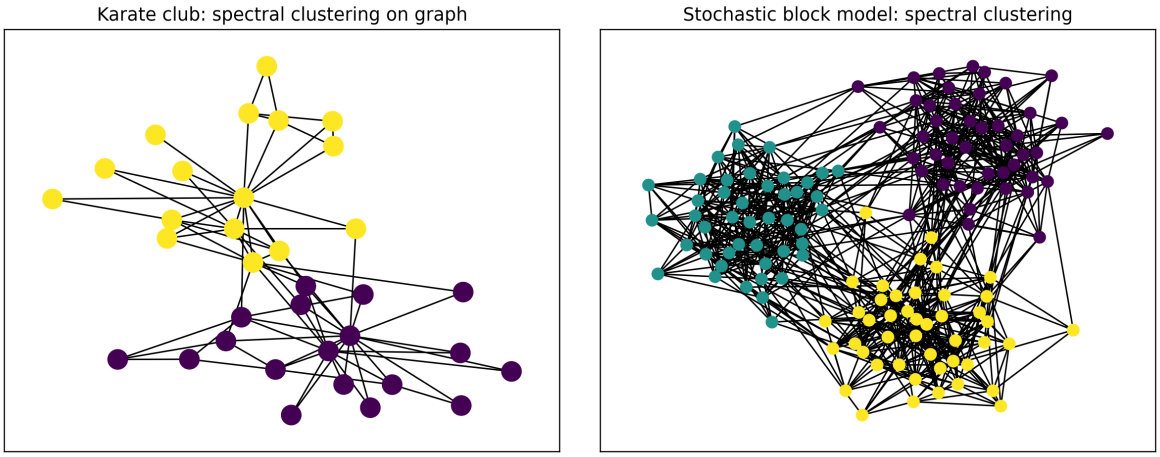

Figure 7: Graph experiments: Zachary's karate club (left) and a stochastic block model benchmark (right).

8 Noise robustness and computational scaling

Figure **??** shows ARI under additive Gaussian noise on two moons. Spectral clustering is excellent at low noise, but performance becomes unstable once noise disrupts the local graph structure. The curve is not monotone: the small bump around intermediate noise is likely seed-level variation from a single trial, and should not be over-interpreted. The robust conclusion is that the method has a low-noise success regime and then rapidly becomes graph-construction sensitive.

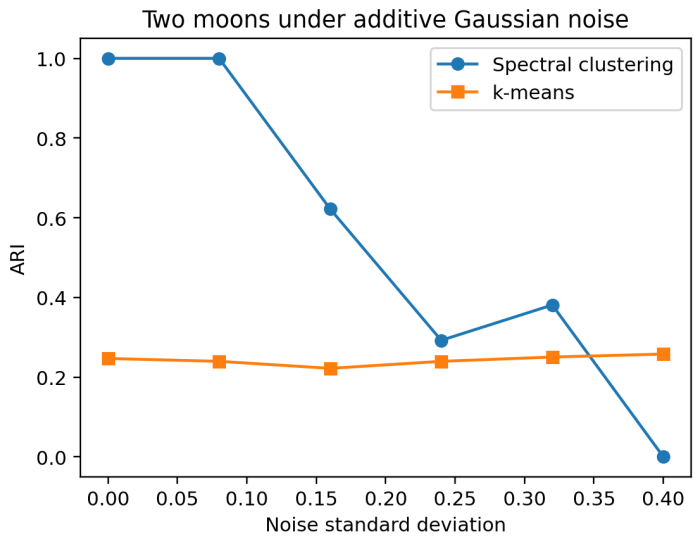

Figure 8: ARI versus additive Gaussian noise on two moons. Spectral clustering succeeds at low noise, then becomes unstable once the neighborhood graph no longer reliably tracks the moon geometry.

Figure **??** reports runtime and approximate memory scaling. The plot makes the main computational lesson visually obvious: once the graph is built, the eigensolver is the expensive stage, while the final k-means step in spectral space is relatively small.

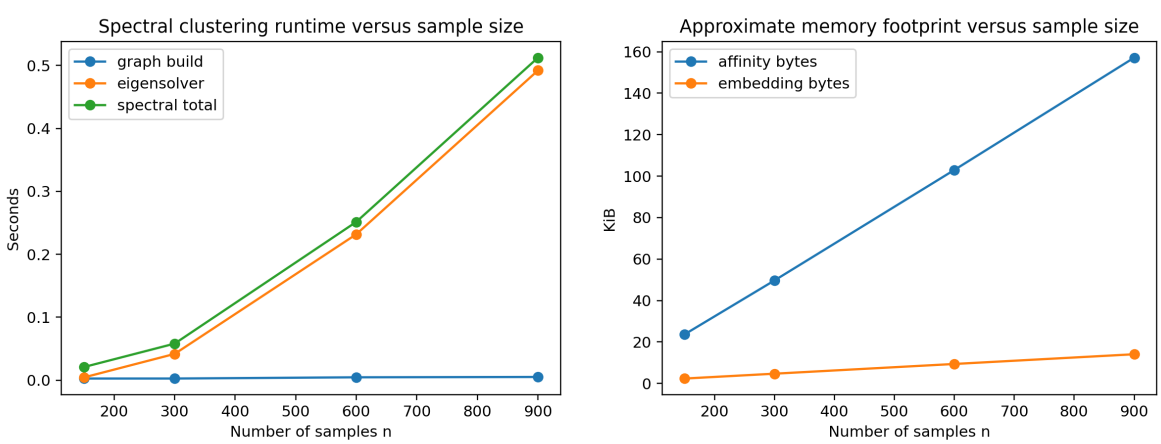

Figure 9: Runtime and approximate memory scaling within the spectral pipeline on a two-moons benchmark.

9 Additional experiments

9.1 Parameter sensitivity map

Spectral clustering depends strongly on the graph it constructs. Figure **??** shows a phase-diagram style sweep on two moons, varying both the feature noise and the number of nearest neighbors in the graph. The final red-outlined column reports the k-means baseline at the same noise level. The main pattern is useful: there is a broad stable band at moderate neighbor counts, but too few neighbors can fragment the graph, while too many neighbors can wash out the two-moon geometry once noise is substantial. In the high-neighbor regime, performance often approaches the k-means baseline, consistent with the idea that an overly dense neighborhood graph loses the local manifold structure that gives spectral clustering its advantage.

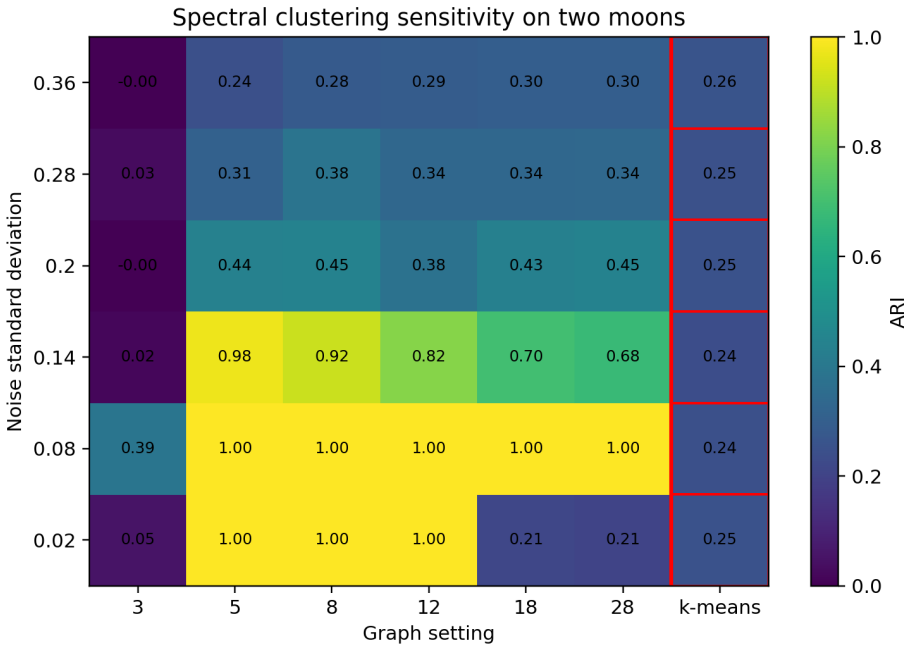

Figure 10: Parameter sensitivity map on two moons. Each cell is ARI for normalized spectral clustering as the noise level and k-NN graph degree vary; the red-outlined column gives the k-means reference value at the same noise level.

Figure **??** makes this more concrete by showing representative rows and columns of the heatmap. Along rows, the low-noise case shows a wide range of graph degrees with near-perfect recovery, while the high-noise case remains poor across the entire sweep. Along columns, increasing noise gradually erodes the local graph signal until the spectral method behaves little better than the centroid baseline.

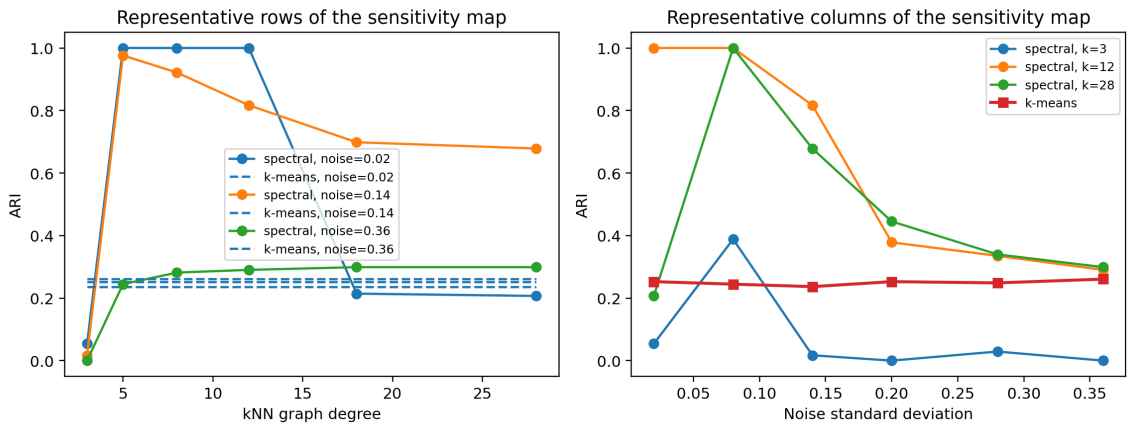

Figure 11: Representative slices of the sensitivity map. Left: ARI as graph degree varies for several fixed noise levels, with dashed k-means references. Right: ARI as noise varies for several fixed graph degrees.

9.2 Eigengap study

A common heuristic for spectral clustering is that the first few eigenvalues of the graph Laplacian reveal the number and strength of clusters. Figure **??** compares the smallest eigenvalues on several datasets. The stochastic block model has a clear gap after the third eigenvalue and is perfectly recovered, which matches the textbook story. The two-moons case also has useful low-eigenvalue structure. But the overlapping blobs and digit cases are less clean: a visible gap alone is not a complete certificate of good clustering. This experiment makes the eigengap heuristic concrete while also showing its limitation.

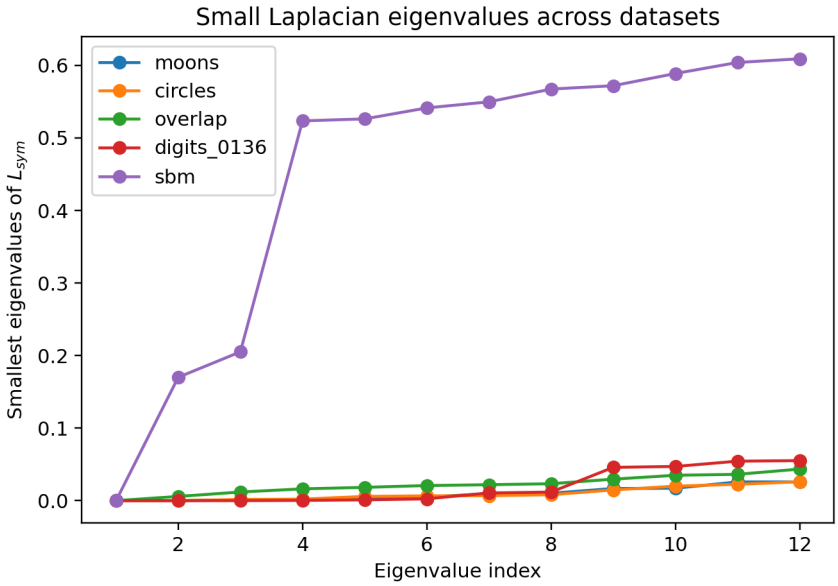

Figure 12: Smallest eigenvalues of the symmetric normalized graph Laplacian. A clean eigengap often correlates with clusterability, but it is not by itself a guarantee of high clustering accuracy.

9.3 Graph construction ablation

The strongest additional lesson is that graph construction is the main inductive bias. We compare three affinity constructions:

$$W_{ij}^{\text{dense}} = \exp(-\gamma\|x_i - x_j\|_2^2),$$

$$W_{ij}^{\text{kNN}} = \mathbf{1}\{j \in \mathcal{N}_m(i) \text{ or } i \in \mathcal{N}_m(j)\} \exp(-\gamma\|x_i - x_j\|_2^2),$$

and

$$W_{ij}^{\text{mutual}} = \mathbf{1}\{j \in \mathcal{N}_m(i) \text{ and } i \in \mathcal{N}_m(j)\} \exp(-\gamma\|x_i - x_j\|_2^2).$$

Here $\mathcal{N}_m(i)$ denotes the directed m-nearest-neighbor set of point i. Figure **??** shows that the same spectral clustering pipeline can behave very differently depending on this choice. On moons and circles, local sparse graphs preserve manifold geometry and dense RBF does poorly. On the digit subset $\{0, 1, 3, 6\}$, dense RBF performs well, while mutual-k-NN becomes too restrictive and loses useful edges. The conclusion is not that one graph is universally best; rather, graph construction must match the data geometry.

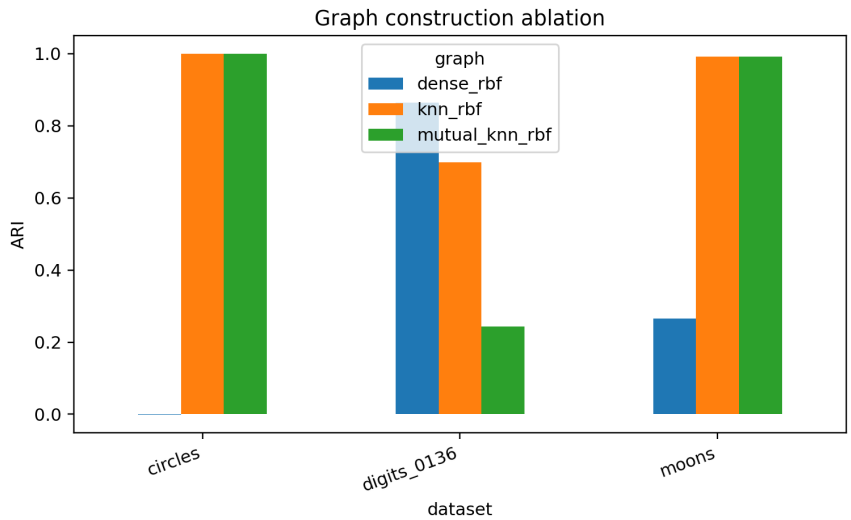

Figure 13: Graph construction ablation. The same spectral clustering pipeline can behave very differently depending on whether the graph is dense, local sparse, or mutual local.

9.4 Graph fragility archetypes

Figure **??** collects three interpretable fragility modes. First, too few neighbors can fragment a manifold graph. Second, a long thin bridge of intermediate points can remove the clean low-density separation between two masses. Third, varying density and imbalance can make a single global neighborhood scale inappropriate. The point of this figure is not that spectral clustering always fails on such inputs; rather, it illustrates how fixed graph parameters can become mismatched to the local geometry of the data.

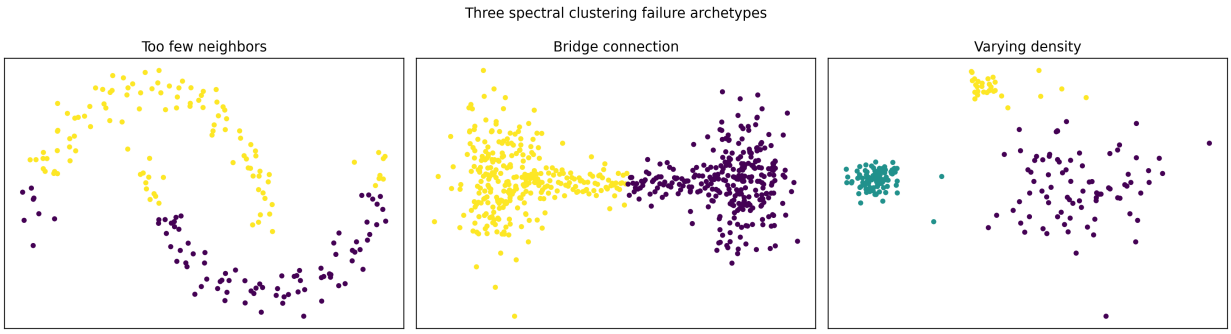

Figure 14: Three graph-fragility archetypes in original 2D coordinates (x, y): graph fragmentation, a long thin bridge, and varying-density geometry.

9.5 Runtime bottleneck breakdown

Figure **??** expands the computational scaling analysis by decomposing spectral clustering runtime into graph construction, eigensolver, and final k-means-on-embedding stages, and overlays the plain k-means baseline from the same benchmark. For the largest sizes tested, the spectral total almost exactly tracks eigensolver time, while the baseline remains much smaller. This agrees with the theoretical discussion: once a sparse graph has been built, the expensive stage is extracting the relevant eigenvectors of a large sparse Laplacian.

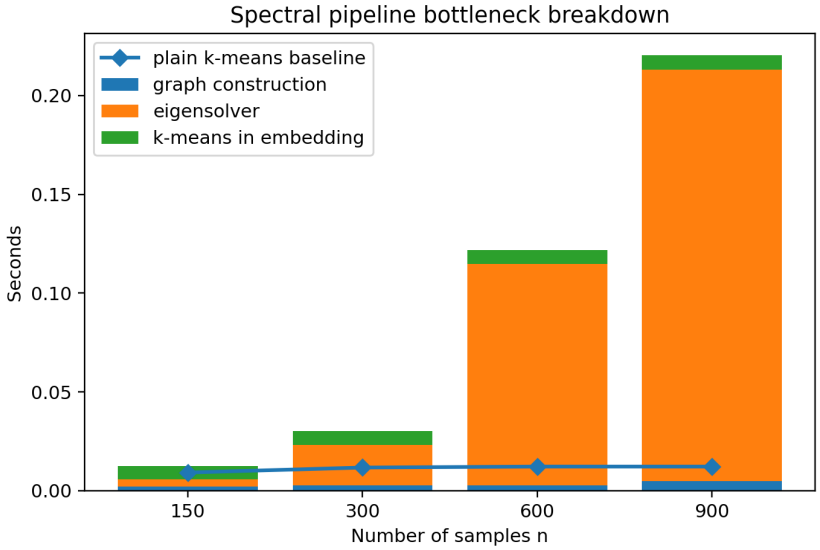

Figure 15: Runtime bottleneck breakdown for the spectral clustering pipeline, with a plain k-means baseline overlaid. The final k-means step is small; the eigensolver becomes the dominant cost as n grows.

10 Discussion

Overall, the experiments match the standard theory quite well.

- On non-convex synthetic datasets, spectral clustering substantially outperforms k-means.

- On standard blob-like data, both methods are strong and k-means can even be slightly better.
- On graph/community datasets, spectral clustering is especially natural and strong.
- On the digit subsets, spectral clustering is not uniformly better; it ties on $\{0, 1\}$, is nearly perfect but slightly worse on $\{1, 7\}$, loses on $\{0, 1, 3, 6\}$, and wins clearly on all ten digits. This is a useful reminder that graph construction is a modeling choice, not a free lunch.

The main surprises are the $\{1, 7\}$ and $\{0, 1, 3, 6\}$ digit subsets, where k-means already aligns well with raw pixel geometry and our particular neighborhood graph is comparatively poor. The high-dimensional moons and bridge examples also show that a numerical advantage can be visually modest. These are informative negative results rather than embarrassments: they expose exactly where normalized spectral clustering depends on a good graph model.

11 Reproducibility

The code is organized as a small Python workspace. Running

```
cd code
python run_all.py
```

regenerates all figures in `final_pic/` and all tables/CSVs in `results/`. The workspace is structured so it can be extended easily with additional datasets or parameter sweeps.